

# PYTHON INTERNSHIP PROJECT

Project Portfolio, Automation & Web Deployment

Submitted By

**Ashwina Pandey**

Roll No.: 25SCS1003004835

B.Tech. CSE • Section 2CSE4

**IILM University**

Academic Year: 2025–2029

## Python Internship Projects

A collection of four completed Python deliverables

### Python Sandbox

Configured a Python virtual environment and created an entry-point script with logging and verification.

Completed

### Directory Cleanup

Automatically scans files and organizes documents and images into folders based on their extensions.

Completed

### Image Downloader

Reads image URLs from a text file and downloads multiple images using Python requests.

Completed

### Cron Scheduler & Logger

Simulates scheduled task executions and records timestamps and execution statistics in a log file.

Completed

Python Internship Deliverables • Project Portfolio

# Project Overview

OVERVIEW

## Python Sandbox

Isolated Python environment, entry-point script and verification logging.

## Directory Cleanup

Automated organization of documents and images by file extension.

## Image Downloader

Reads image URLs and downloads multiple images using requests.

## Cron Scheduler & Logger

Simulated scheduled executions with timestamps and execution statistics.

### Python Internship Projects

A collection of four completed Python deliverables

#### Python Sandbox

Configured a Python virtual environment and created an entry-point script with logging and verification.

Completed

#### Directory Cleanup

Automatically scans files and organizes documents and images into folders based on their extensions.

Completed

#### Image Downloader

Reads image URLs from a text file and downloads multiple images using Python requests.

Completed

#### Cron Scheduler & Logger

Simulates scheduled task executions and records timestamps and execution statistics in a log file.

Completed

Python Internship Deliverables • Project Portfolio

Figure 2.1: Project portfolio web interface showing all four completed deliverables.

# Objectives

- Build practical Python programs using core libraries and modules.
- Understand virtual environments, dependencies and project structure.
- Automate file organization and image downloading tasks.
- Implement task scheduling concepts and persistent execution logging.
- Develop a Flask-based web interface for the completed deliverables.
- Use Git and GitHub for version control and project hosting.
- Deploy the web application to a public cloud platform using Render.
- Practice testing, debugging, documentation and software delivery.

# Technologies Used

OVERVIEW

## Python

Core programming language

## Flask

Web application framework

## Requests

HTTP requests & image downloads

## OS / Shutil

File and directory operations

## Logging / Time

Execution records & timing

## Git / GitHub

Version control & repository

## Gunicorn

Production WSGI server

## Render

Cloud deployment

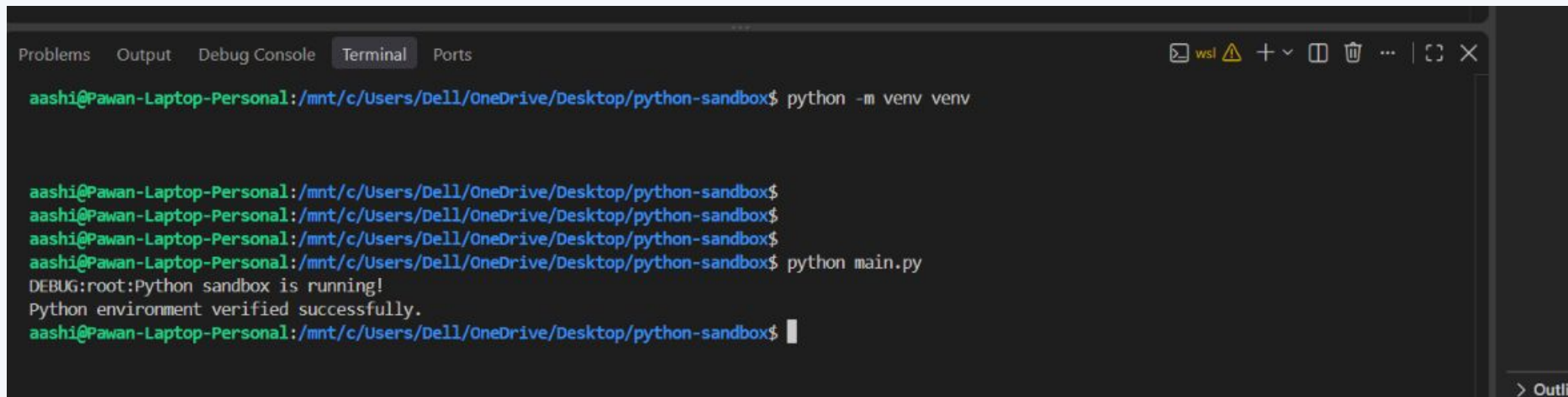
# Task 1 — Python Sandbox Setup

- Created an isolated Python virtual environment for the project.
- Used logging to verify that the environment and entry-point script were working.
- Maintained dependencies through requirements.txt.
- Verified execution successfully from the terminal.

A screenshot of a code editor showing the content of a file named 'main.py'. The code is written in Python and includes logging configuration and two log messages. The lines are numbered 1 through 6.

```
main.py
1  import logging
2
3  logging.basicConfig(level=logging.DEBUG)
4
5  logging.debug("Python sandbox is running!")
6  logging.info("Python environment verified successfully.")
```

Figure 5.1: main.py entry-point and logging configuration.

A screenshot of a terminal window showing the execution of a Python script. The terminal has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active. The user 'aashi' is in a WSL environment on a Dell laptop. The terminal shows the creation of a virtual environment, activation, and the execution of 'python main.py', which outputs debug and info messages.

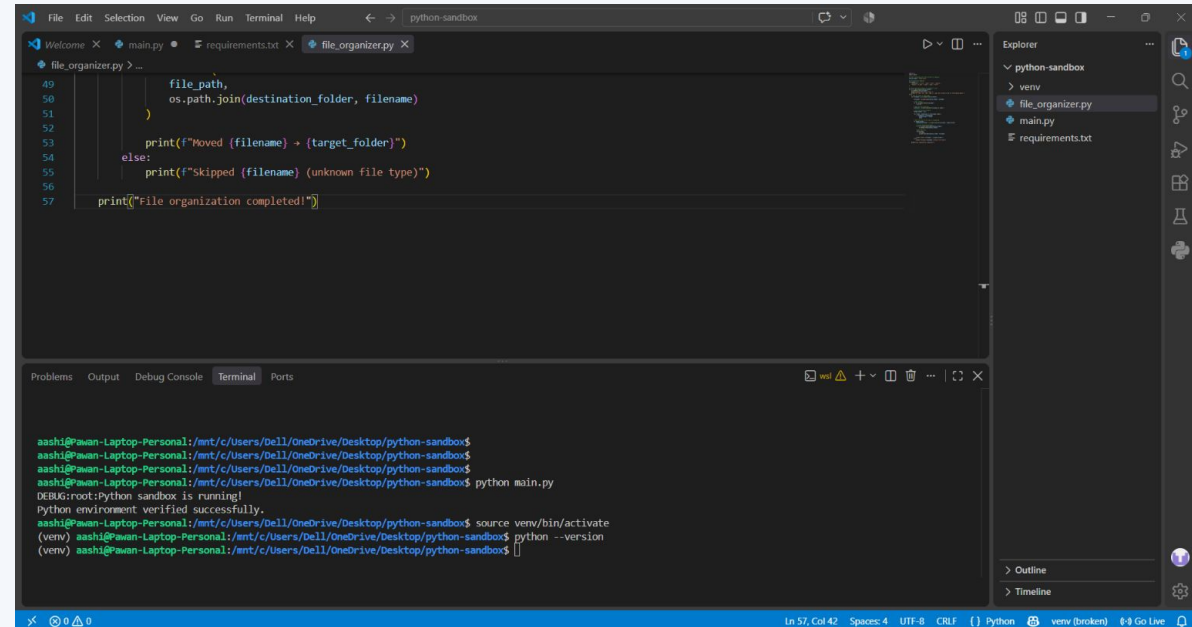
```
Problems  Output  Debug Console  Terminal  Ports
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$ python -m venv venv

aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$ python main.py
DEBUG:root:Python sandbox is running!
Python environment verified successfully.
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$
```

Figure 5.2: Successful Python Sandbox verification in the terminal.

# Task 2 — Directory Cleanup

- Scanned a target directory and identified files by extension.
- Mapped document and image extensions to destination folders.
- Used os and shutil for directory creation and file movement.
- Successfully separated documents and images into organized folders.



```
49     file_path,
50     os.path.join(destination_folder, filename)
51     )
52
53     print(f"Moved {filename} → {target_folder}")
54     else:
55         print(f"Skipped {filename} (unknown file type)")
56
57     print("File organization completed!")
```

```
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$ python main.py
DEBUG:root:Python sandbox is running!
Python environment verified successfully.
aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$ source venv/bin/activate
(venv) aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$ python --version
(venv) aashi@Pawan-Laptop-Personal:/mnt/c/Users/Dell/OneDrive/Desktop/python-sandbox$
```

Figure 6.1: file\_organizer.py implementation.

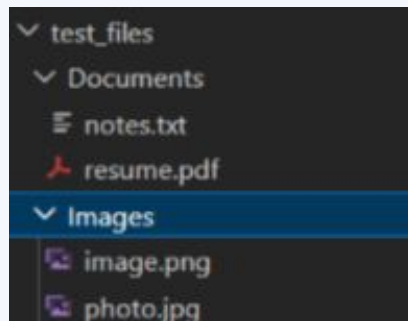


Figure 6.2: Organized Documents and Images folders.

## Automation Flow

Scan files → Detect extension → Create destination → Move file

# Task 3 — Image Downloader

## TASK 3

- Stored image URLs in image\_urls.txt.
- Used requests to retrieve image content over HTTP.
- Used ThreadPoolExecutor to download multiple images efficiently.
- Saved successfully downloaded images in the downloaded\_images folder.
- Handled network and file-related exceptions.

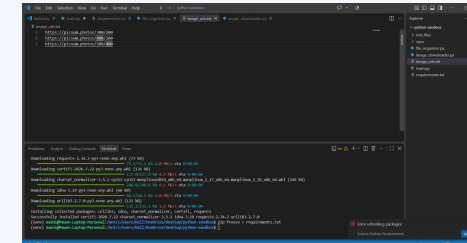


Figure 7.1: Input image URLs.

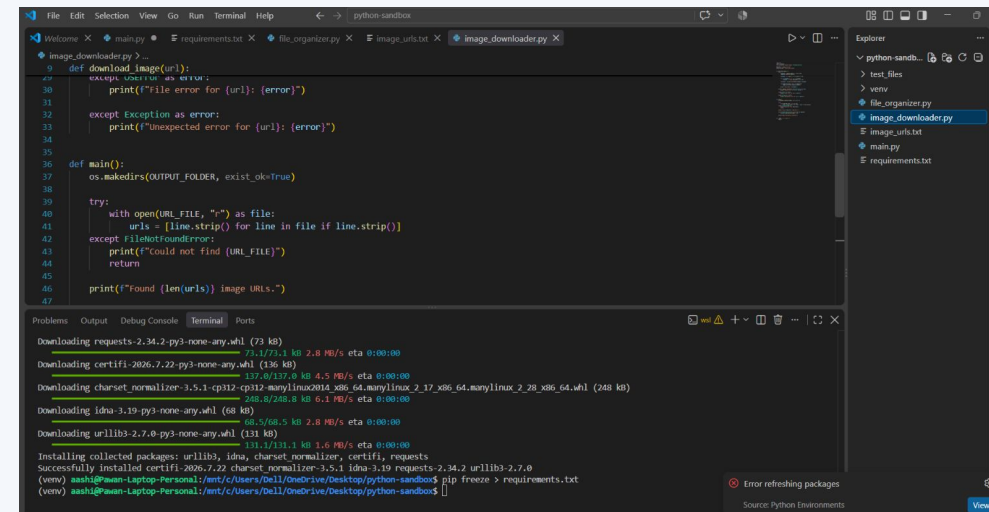


Figure 7.2: Image downloader implementation.

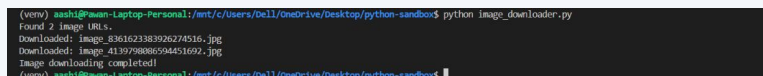


Figure 7.3: Successful image download output.

# Task 4 – Cron Scheduler & Logger

- Simulated three scheduled task runs.
- Recorded run numbers, timestamps and task status.
- Measured execution time for each scheduled task.
- Stored execution history in scheduler.log.
- Confirmed all scheduled runs completed successfully.

```

1 scheduler_logger.py >..
2 import logging
3 import time
4 from datetime import datetime
5
6
7 # configure logging
8 logging.basicConfig(
9     filename="scheduler.log",
10    level=logging.INFO,
11    format="%(asctime)s - %(levelname)s - %(message)s"
12)
13
14
15 def scheduled_task():
16     """simulate an automated task."""
17     start_time = time.time()
18
19     logging.info("Scheduled task started.")
20
21     # Simulated task work
22     time.sleep(1)
23
24     end_time = time.time()
25     execution_time = end_time - start_time
26
27     logging.info(
28         f"Scheduled task completed successfully. "
29         f"Execution time: {execution_time:.2f} seconds"
30     )
31
32     print(
33         f"Task executed at {datetime.now().strftime('%Y-%m-%d %H:%M:%S')} "
34         f"in {execution_time:.2f} seconds."
35     )
36

```

Figure 8.1: Scheduler and logging implementation.

```

1 scheduler.log
2 2020-08-31 13:00:24,000 - INFO - Run #1 initiated.
3 2020-08-31 13:00:24,001 - INFO - Scheduled task started.
4 2020-08-31 13:00:25,002 - INFO - Scheduled task completed successfully. Execution time: 1.00 seconds
5 2020-08-31 13:00:25,002 - INFO - Run #1 finished.
6 2020-08-31 13:00:26,007 - INFO - Run #2 initiated.
7 2020-08-31 13:00:26,008 - INFO - Scheduled task started.
8 2020-08-31 13:00:27,002 - INFO - Scheduled task completed successfully. Execution time: 1.00 seconds
9 2020-08-31 13:00:27,002 - INFO - Run #2 finished.
10 2020-08-31 13:00:28,008 - INFO - Run #3 initiated.
11 2020-08-31 13:00:28,009 - INFO - Scheduled task started.
12 2020-08-31 13:00:29,008 - INFO - Scheduled task completed successfully. Execution time: 1.00 seconds
13 2020-08-31 13:00:29,008 - INFO - Run #3 finished.
14 2020-08-31 13:00:29,010 - INFO - All scheduled runs completed.

(venv) ash@Pawen-Laptop-Personal:~/nt/c/Users/ball/OneDrive/Desktop/python-sandbox$ python image_downloader.py
Traceback (most recent call last):
  File "image_downloader.py", line 1, in <module>
    import urllib
ModuleNotFoundError: No module named 'urllib'

(venv) ash@Pawen-Laptop-Personal:~/nt/c/Users/ball/OneDrive/Desktop/python-sandbox$ python scheduler_logger.py
scheduler.log: command not found

(venv) ash@Pawen-Laptop-Personal:~/nt/c/Users/ball/OneDrive/Desktop/python-sandbox$ python scheduler_logger.py
Cron Scheduler & Logger started.
Task executed at 2020-08-31 13:00:25 in 1.00 seconds.
Task executed at 2020-08-31 13:00:29 in 1.00 seconds.
Task executed at 2020-08-31 13:00:32 in 1.00 seconds.
All scheduled runs completed.
(venv) ash@Pawen-Laptop-Personal:~/nt/c/Users/ball/OneDrive/Desktop/python-sandbox$

```

Figure 8.2: Scheduler log and terminal output showing three successful runs.



# Web Application

- Developed a Flask-based project portfolio interface.
- Displayed all four completed Python deliverables.
- Styled the interface using HTML and CSS.
- Tested the application locally before deployment.
- Configured the application to run on the required host and port.

```

1 from flask import Flask
2
3 app = Flask(__name__)
4
5
6 @app.route("/")
7 def home():
8     return """
9     <doctype html>
10     <html>
11     <head>
12         <title>Python Internship Projects</title>
13         <style>
14             body {
15                 font-family: Arial, sans-serif;
16                 background-color: #f4f2f8;
17                 margin: 0;
18                 padding: 40px;
19                 color: #1f2937;
20             }
21
22             .container {
23                 max-width: 800px;
24                 margin: auto;
25             }
26
27             .h1 {
28                 text-align: center;
29                 margin-bottom: 10px;
30             }
31
32             .subtitle {
33                 text-align: center;
34                 color: #667788;
35                 margin-bottom: 10px;
36             }

```

Figure 9.1: Flask application code.

The screenshot shows a local development environment. On the right, a file explorer lists files like `python-sandbox`, `downloaded_images`, `test_files`, `new`, `gltignore`, `app.py`, `file_organizer.py`, `image_downloader.py`, `image_url.txt`, `main.py`, `requirements.txt`, `schedule_logger.py`, and `schedule_log`. The main editor shows `requirements.txt` with the following content:

```

1 flask
2 pyyaml
3

```

The terminal at the bottom shows the following output:

```

Successfully installed blinker-1.9.0 click-8.0.0 flask-3.1.0 itsdangerous-2.2.0 Jinja2-3.1.0 MarkupSafe-3.0.0 Werkzeug-3.1.0
(venv) ashi@Aman-Laptop-Personal:~/c:/Users/Dell/OneDrive/Desktop/python-sandbox$ pip freeze > requirements.txt
(venv) ashi@Aman-Laptop-Personal:~/c:/Users/Dell/OneDrive/Desktop/python-sandbox$ python app.py
(venv) ashi@Aman-Laptop-Personal:~/c:/Users/Dell/OneDrive/Desktop/python-sandbox$ python app.py
* Serving Flask app "app"
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.26.179.240:5000
Press CTRL+C to quit
127.0.0.1 - - [21/Aug/2025 13:09:10] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [21/Aug/2025 13:09:10] "GET /favicon.ico HTTP/1.1" 404 -
(venv) ashi@Aman-Laptop-Personal:~/c:/Users/Dell/OneDrive/Desktop/python-sandbox$

```

Figure 9.2: Local Flask application running successfully.

# GitHub Version Control

- Initialized and maintained the project using Git.
- Committed project changes with meaningful commit messages.
- Pushed the main branch to GitHub.
- Repository is public and contains the completed source files.
- GitHub provides a clear version history and project structure.

## Repository

[github.com/ashwina0202-ai/python-internship-project](https://github.com/ashwina0202-ai/python-internship-project)

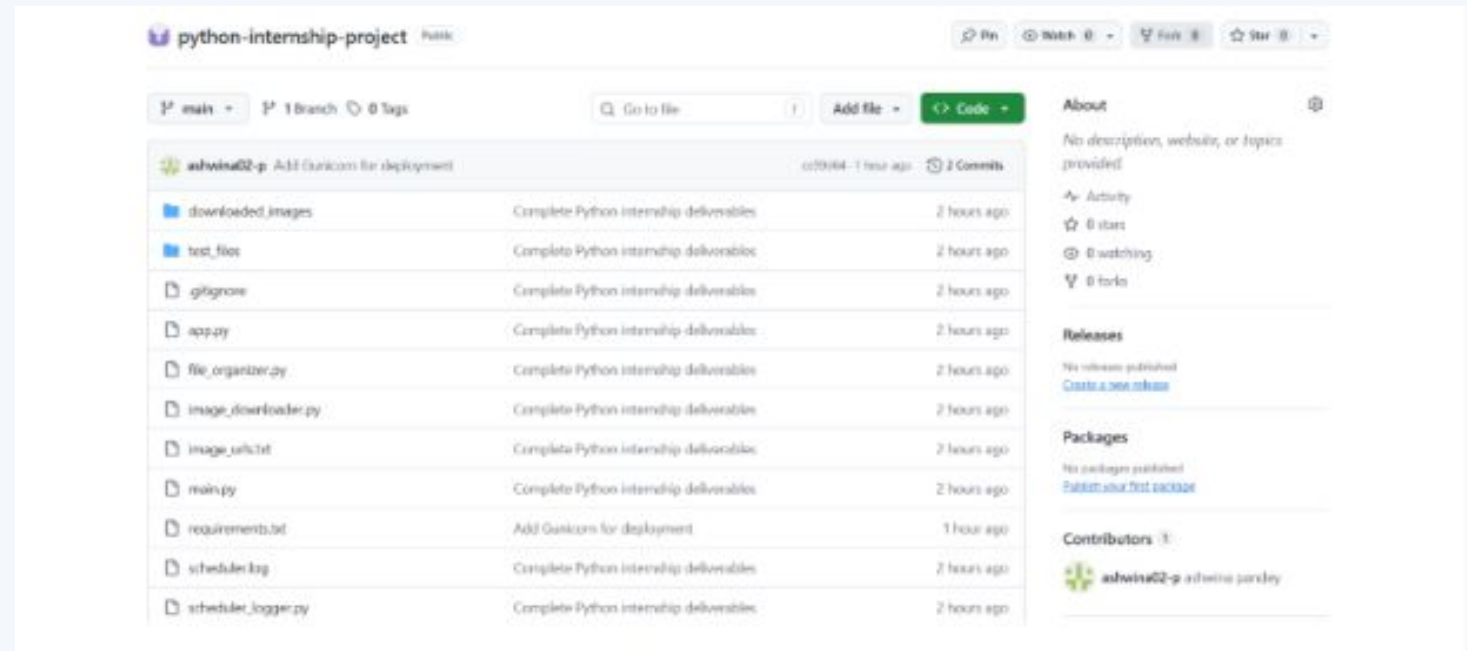


Figure 10.1: Public GitHub repository containing the internship project.

# Live Deployment

- Configured Gunicorn as the production WSGI server.
- Connected the Render service to the GitHub repository.
- Deployed the Flask application as a public web service.
- Verified the live application after deployment.

## Live Application

[python-internship-project-m0ny.onrender.com](https://python-internship-project-m0ny.onrender.com)

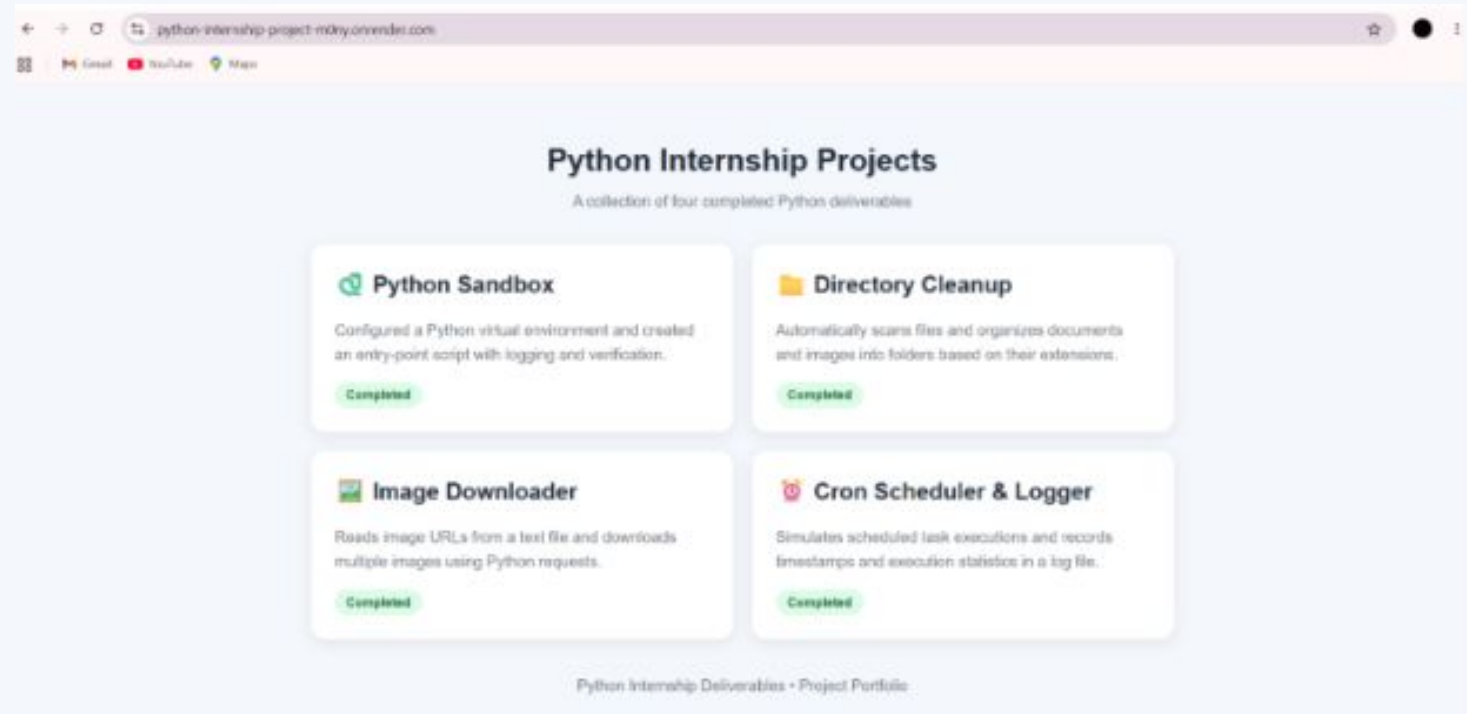


Figure 11.1: Live Python Internship Projects application deployed on Render.

# Testing & Results

RESULTS

## Python Sandbox

Environment verified successfully

## Directory Cleanup

Files organized by extension

## Image Downloader

Multiple images downloaded successfully

## Scheduler & Logger

Three scheduled runs completed and logged

## Flask Web App

Local application tested successfully

## GitHub

Source code pushed to public repository

## Render

Live application deployed successfully

# Conclusion & Future Scope

## CONCLUSION

### Conclusion

The internship project provided practical experience across Python programming, automation, file handling, HTTP requests, logging, Flask web development, Git/GitHub, testing and cloud deployment.

All four core deliverables were implemented, tested and presented through a live web application.

### Future Scope

- Add a database for execution history
- Add authentication and interactive controls
- Add real-time monitoring and notifications
- Expand file-type support and testing
- Improve analytics and reporting

# INTERNSHIP CERTIFICATE



# THANK YOU

## Python Internship Project

**Ashwina Pandey**

25SCS1003004835 • B.Tech. CSE • 2CSE4

IILM University

GitHub: [github.com/ashwina0202-ai/python-internship-project](https://github.com/ashwina0202-ai/python-internship-project)

Live: [python-internship-project-m0ny.onrender.com](https://python-internship-project-m0ny.onrender.com)